The Bonfire

The Bonfire
Faction Management

Leadership operations, player contribution and ranked-war payouts.

Version 1.2 / 1 November 2026
Implementation specification and interactive prototype contract

Audience: faction leaders, officers, treasurers and the developer implementing the extension. The existing Torn Chain Tracker is version one; this specification extends it rather than treating it as a new product.

Confirmed policy: allocate 10% of confirmed war income to supplies, 10% to other faction expenses and 80% to players. Players receive Xanax for free. Xanax is a faction-reserve expense, never a second deduction from the player pool.

Confirmed constraints: no additional keys or actions from ordinary players; no private battle/work-stat module; preserve the current live site during prototyping. All UI and requirements in this package are in English.

Payment discovery: Torn supports a link that preselects a member and amount in Give to User. The actual transfer still requires confirmation in Torn. Pay Day has different member-balance effects and does not have a verified prefill contract in this specification.

This version incorporates calculator clarity, internal chain reports, separate payment workflow, official war timing and key setup. It replaces the earlier requirements. Proposed defaults are clearly separated from confirmed faction policy. No live transfers or private API-key tests were performed.

Scope and delivery map

Pages	Coverage
03-04	Existing baseline, access roles and metric definitions.
05-09	Reconstructed spreadsheet, funds, calculation modes, six presets and simple editing.
10-13	Approval lifecycle, overview, players, wars, chains, payments and reconciliation.
14	Pay in Torn: verified prefill, Pay Day distinction and payment-intent lifecycle.
15-17	API mapping, synchronization, data model, privacy and reliability.
18-19	Rounding, edge cases and numerical acceptance tests.
20-21	Migration, priorities and interactive prototype coverage.
22-23	Official war timing, active-war discovery and faction-key setup.
24	Confirmed decisions, open production checks and sources.

Primary outcome

An officer opens a war, verifies its contribution records, selects a payout preset and edits a few values. A leader approves an explainable settlement. A treasurer prepares a transfer in Torn and records or reconciles its completion without paying twice.

Required extension

Faction overview; player table and profile; chain history/details; ranked-war list/details; editable calculation rules and presets; settlement periods; payment ledger; manual adjustments; source coverage; role-based permissions; audit history; CSV export; payment-form links.

Follow-on capabilities

Chain-watcher shifts, bonus-hit scheduling and configurable in-app alerts. Discord is optional and must not be required for the core product.

Excluded

Executing attacks, sending money via the read-only Torn API, automated browser transfers, selling rewards, real-currency accounting, and private player battle/work stats. Do not request personal keys from members to expand the feature set. Monetary amounts are whole Torn dollars, denoted $.

Existing baseline and access roles

The inspected V1 interface contains a member leaderboard, hits/respect/bonus hits, search/sort, event settings, chain-report and CSV imports, chain history, CSV export and destructive cleanup controls. The backend was not supplied. Verify its actual behavior before migrating data.

Role	Permissions
Leader / Admin	Manage access, faction key and sync; maintain presets; approve/revise settlements; view audit records.
Officer / Analyst	Read permitted data, prepare drafts, annotate records and propose adjustments. Approval requires a separate permission.
Treasurer	Work with approved obligations, prepare payment links, record/reconcile transfers, reverse incorrect ledger entries and export.
Viewer	Read permitted views. Financial amounts and private notes are separately permissioned.

AUTH-01. Enforce permissions on every server operation. Hidden or disabled controls are not authorization. One person may have multiple roles.

AUTH-02. The existing shared password may remain a temporary read-only entry point. Mutations, approvals and payment records require individual leadership identities for a meaningful audit trail. This does not require accounts or keys from ordinary members.

AUTH-03. Never expose Torn keys in client code, logs, exports, URLs or this document. Store the faction credential on the server. A Torn login is used only on torn.com, never collected by this product.

AUTH-04. Archive instead of cascade-deleting members with contributions, obligations or payments. Move cleanup away from the main workspace. Preserve names as historical labels and Torn player_id as identity.

Contribution definitions

Metric	Product definition
War hits W	Successful outgoing attacks against the opponent of the selected ranked war, within its time window. Validate result, opponent faction and war context.
Outside hits N	Other successful outgoing faction attacks in the same window. Visible even when their payout weight is zero.
Assists A	A distinct category with result Assist. Never also count the same record as W or N.
Total hits	W + N. This is not all attack attempts.
Total attacks	All relevant outgoing attempts, including failures, assists and interrupted attempts. Informational unless a rule explicitly uses it.
Respect R	Actual respect gained from eligible successful hits. Preserve source precision and explicitly define inclusion of chain-bonus respect.
Bonus hits	Confirmed chain milestones. A property of an existing hit, not another attack.
Quota	A target for W, W+N or points. Quota completion is independent of the payout formula.

MET-01. Default successful results: Attacked, Mugged and Hospitalized. Unclassified special results require review. Lost, Escape, Stalemate, Timeout, incoming and Interrupted records are unpaid by default.

MET-02. Use ended timestamp: war.start <= ended <= war.end. Adjacent arbitrary settlement windows must not own the same payable attack twice. Store time in UTC; show TCT/UTC or an explicitly labelled local zone.

MET-03. Do not rely on the experimental is_interrupted flag alone to determine a payable assist. Reconcile totals with the ranked-war report; expose differences rather than silently forcing a match.

Spreadsheet reconstruction

The user supplied an image of the current spreadsheet for 28 August 2026. The original formula file was not supplied. Visible nonzero rows reproduce the following calculation.

Input or result	Value
Confirmed income	182,285,770
Allocation	10% supplies / 10% other expenses / 80% players
Player pool	145,828,616
Weights W / N / A / R	1 / 0 / 0 / 0.1
Visible war hits	160
Visible respect	1,019.52
Respect points	101.952
Total points	261.952
Rate per point	145,828,616 / 261.952 = 556,699.761788…

$$P_i = W_i + 0.1 \times R_i$$
$$Pay_i = 145{,}828{,}616 \times P_i / 261.952$$

OZONE03: 13 W and 83.75 R produce 21.375 points and an unrounded share of 11,899,457.408… . The displayed payout is approximately $11,899,457 before the deterministic remainder-allocation policy. Hit Pay and Respect Pay may differ by $1 when rounded independently.

Misleading labels: the screenshot shows 262.0 under Hit point and 102 under Respect point. These are rounded displays of 261.952 total points and 101.952 respect points. Rename them and never use display-rounded numbers as formula inputs.

Xanax $8,300,000: record against the faction reserve, leaving the player pool unchanged. Do not infer that other bank figures on the image belong to this war without an explicit link.

Income, reserves and expenses

FIN-01. Store confirmed income I separately from estimated reward value. Each income entry has a source, date and confirmation status. Reward items do not become cash automatically; proceeds must be confirmed.

Supplies S = floor(I × 0.10)
Other reserve O = floor(I × 0.10)
Player budget B = I - S - O

The player budget receives the possible whole-dollar allocation remainder. For the sample: S = O = 18,228,577 and B = 145,828,616. Display percentages and actual amounts together.

FIN-02. Default 10/10/80 comes from the selected preset. Advanced allocation edits apply only to this war; percentages must total 100%. A manual chain budget replaces income allocation rather than stacking on top of it.

FIN-03. An allocation is not an expense. Supplies, free Xanax and other expenses reduce their chosen reserve, not the player budget again. Show reserve deficits and their explicit funding source.

FIN-04. Separate faction top-up T, inside-pool bonuses J_inside and extra-funded bonuses J_extra. Every extra-funded bonus has a reserve/bank source. All bonuses are zero by default.

Player fund F = B + T
Variable distribution V = F - SUM(J_inside)
Total obligation = F + SUM(J_extra)

FIN-05. Reserve closing balance = opening balance + allocations + other inflows - expenses - outgoing transfers. Keep this internal ledger separate from Torn balance snapshots; reconcile differences explicitly.

No automatic Xanax deductions or penalties. Additional reimbursements outside free supplies require a named policy, explanation and funding source.

Calculation modes

CALC-01. Weighted distribution of a fixed pool

$$P_i = wW \times W_i + wN \times N_i + wA \times A_i + wR \times R_i$$
$$P = \text{SUM}(P_i);\ \text{rate} = V / P$$
$$\text{Base}_i = V \times P_i / P$$
$$\text{RawPay}_i = \text{Base}_i + J_{\text{inside},i} + J_{\text{extra},i}$$

Weights are points per unit, not dollars per attack. Changing one player's eligible contribution changes the rate and other players' payouts. Preview that effect before saving.

CALC-02. Respect scope is explicit: successful war hits only or all successful hits in the selected window. The spreadsheet preset defaults to all successful hits, including chain-bonus respect. This is a proposed interpretation to verify against the full original sheet.

CALC-03. Excluding chain-bonus respect requires confirmed bonus-to-hit/source matching. Subtract verified bonus respect only. Do not infer a bonus from unusually high respect; block approval under that policy if coverage is insufficient.

CALC-04. Fixed dollar rates

$$\text{Base}_i = rW \times W_i + rN \times N_i + rA \times A_i + rR \times R_i$$
$$\text{RawPay}_i = \text{Base}_i + J_{\text{inside},i} + J_{\text{extra},i}$$

Rates are dollars per unit. Total obligations may differ from F. Show available funds, allocated amount and remaining balance or deficit. A deficit blocks approval until rules or funding change. Never silently scale the rates.

CALC-05. An unused fixed-rate balance stays unallocated until an explicit return/reserve action. Excluding a player from payout requires a reason but does not erase their contribution statistics.

Six editable presets

A preset is a versioned template. Applying one creates an independent rule snapshot for the selected war. Editing it never changes other wars or the original template.

Preset	Initial rules	Use
Bonfire 10/10/80	Pool; W=1, N=0, A=0, R=0.1; all-hit respect, bonuses included.	Default; reconstructed spreadsheet policy.
War hits only	Pool; W=1; N=A=R=0.	Simple successful war-hit contribution.
War + support	Pool; W=1, N=0.25, A=0.5, R=0.	Reward outside hits and assists.
Respect performance	Pool; only R=1; war respect; verified chain bonuses excluded.	Reward respect without milestone advantage.
Chain contribution	Manual budget; W=N=1, A=R=0; selected chain window.	Non-war chain event.
Fixed rates	Dollar-rate mode; all rates start at 0 and require positive configured rates.	Guaranteed per-action rewards, subject to funding.

PRE-01. Ranked-war presets default to 10/10/80. Show only nonzero weights/rates and a short description in the chooser. Respect performance is unavailable for approval without the required detail.

PRE-02. Actions: Apply, Save as new preset, Restore preset values. Applying over edited rules shows a change summary. Preserve player adjustments unless explicitly cleared.

PRE-03. Show Modified for this war when appropriate. Saving a global template requires a name and permission. Proposed support weights are suggestions, not confirmed faction policy.

Simple rule editing

UX-01. Open war → select preset → enter confirmed income or manual fund → optionally edit four weights → inspect results. No formula builder or code is required.

UX-02. Keep Payout method visible above the four inputs. Pool mode labels weights in points and shows the effective dollar price of every action beside the input and in a rate summary. Fixed mode labels each editable input $ per action. Preserve independent values for both modes; never reinterpret 1 point as $1 on switching.

UX-03. Advanced contains respect scope, bonus-respect inclusion, top-ups, exclusions and fixed bonuses. Surface a concise summary of all non-default advanced settings.

UX-04. Show why a weight becomes money: effective price per action = action weight × variable fund / total contribution points. With the sample, 1 war hit earns about $556,699.76 and 1 respect about $55,669.98. Explain that implied prices change with fund/contributions; fixed rates do not. Use full precision in final allocation.

UX-05. Recalculate valid drafts immediately; target under 300 ms for 100 players. An empty or invalid field is not zero. Show an inline error, retain the last valid preview and disable save/approval.

UX-06. Editing is separate from saving. Show unsaved state, Save draft and Discard. Warn on leaving a modified form; reopen the last saved revision. Applying a preset affects only this war.

UX-07. The primary war action is Review calculation, then Approve calculation - no transfer. Show the recipient count, funding and owed amounts. Explicitly state that approval sends no money, creates no paid ledger records, and opens the separate payment queue. Filters never alter the calculation population.

UX-08. Click a payout for its explanation: counts → weights/rates → points → base share → bonus → rounding → final. Manual adjustments show reason, author and funding source.

UX-09. The selected visual is concept 1: dark narrow navigation, light workspace, contribution table and pale-teal rule panel. Use the exact product name The Bonfire Faction Management. Correct mock-image arithmetic in the working prototype.

Settlement lifecycle

State	Behavior
Collecting	War ongoing or required records incomplete; estimates are visible, approval blocked.
Draft	Editable rules, funding and adjustments; data readiness checked independently.
In review	A saved candidate carries source hash and rule version; any edit returns it to draft.
Approved	Immutable calculation/obligation snapshot. Later sync never silently rewrites it.
Partially paid	At least one recorded payment and an outstanding balance.
Paid	All obligations satisfied or explicitly resolved through an approved adjustment.
Archived	Read-only history retaining every calculation revision and transfer record.

FLOW-01. Torn war status is distinct from local settlement status. Keep previous revisions but permit one active approved revision.

FLOW-02. Before any payment, an authorized leader may withdraw approval with a reason. After payments, create a new revision and reconcile obligation differences; do not erase old payments.

FLOW-03. If revised entitlement falls below paid amount, show an overpayment. Do not create negative cash transfers or silently offset a different war.

FLOW-04. A period can include several chains/wars but each payable attack or aggregate coverage unit has one active settlement owner. A chain nested inside a war is not paid twice by default.

FLOW-05. Approval atomically checks revision, source hash, funding and permission. Concurrent edits or new sync invalidate a stale preview.

Faction overview and players

OV-01. Overview shows selected period, active wars, unpaid obligations, participation, target progress and source issues. Administrative import/cleanup forms do not occupy the main workspace.

OV-02. Show participants out of the relevant roster only when roster coverage is known. Distinguish did not participate, not eligible and missing data.

PL-01. Player table supports name/ID, W, N, assists, total hits/attempts, respect, bonus hits, quota completion, points, accrued/paid/outstanding amounts. Keep essential columns visible and expose additional columns on demand.

PL-02. Search name/ID; sort and filter by war, chain, period, membership and status. Useful filters: below quota, no contribution, unpaid, adjusted and needs data review. Filtering changes displayed rows, never owed amounts.

PL-03. Player detail shows contribution by war/chain, quota history, payout breakdowns, payment history and permissioned notes. Former members retain their obligations and history.

PL-04. Compare equal completed periods or explicitly selected events. Show baseline, absolute change and percentage-point changes for rates. Label partial periods and different populations.

PL-05. Quotas may be faction-wide or individual with effective dates and exceptions. New members and approved absence are not automatically labelled underperformers. Quotas do not imply penalties.

PL-06. Last action is a current-status indicator, not proof of hours worked. No battle/work-stat collection or personal-key onboarding. Progress means contribution, participation and payout progress.

Wars, chains and coordination

CH-01. View in-app report opens a full internal chain view with searchable members, hits/share, regular and bonus respect, total respect, bonus-hit counts and CSV. A separate Bonus hits tab lists milestone, player, reward and real timestamp where available; Sources explains coverage. External Torn report is an optional secondary source link, never the main report.

CH-02. Active chain shows current count, timeout, next milestone, distance and cooldown. Show snapshot time; a local countdown is an estimate. Stale data must not look live.

WAR-01. War list includes opponent, dates, result, rewards, confirmed income, source readiness, settlement state and remaining obligation. Item values are estimates until confirmed.

WAR-02. Wars owns official timing, contributions, payout rules and calculation approval. Payments owns only recipient obligations, individual transfers, pending API confirmation, paid/due amounts and transfer history. Do not duplicate the payment table inside Wars or repeat income/reserve calculations in Payments. Link from an approved calculation to its payment queue.

Follow-on coordination

CO-01. Watcher slots have primary/reserve, confirmation, uncovered times and conflict checks. Store UTC; display TCT and local zone explicitly. Daylight-saving transitions must not duplicate slots.

CO-02. Milestone plan records assigned and actual hitter with replacement reason. Torn report establishes actual outcome; assignment itself never creates a payout.

CO-03. Alert rules cover low timeout, approaching bonus, shift start, uncovered slot and sync failure. Configure thresholds, audience, quiet hours, repetition and deduplication.

CO-04. Start with in-app alerts. Discord is optional with test delivery. With stale snapshots, alert about stale data rather than asserting a chain broke. No guarantee that an alert can prevent a break.

Payments and reconciliation

Paid_i = sum of posted, non-reversed payments
Due_i = max(Approved_i - Paid_i, 0)
Overpaid_i = max(Paid_i - Approved_i, 0)

PAY-01. Ledger columns: recipient, approved, paid, due, payment state, last payment and adjustment reason. Label filtered subtotal separately from the full settlement total.

PAY-02. A payment record requires member ID, positive whole amount, date, source/method and evidence/reference. Partial payments are supported. Record full payment shows the exact remaining balance before confirming the fact of transfer.

PAY-03. Repeated requests with the same idempotency key return the same ledger item. A source transaction cannot be fully allocated twice. Split allocations must not exceed its total.

PAY-04. Automatically poll faction giveFunds news after preparation and during scheduled sync. Confirm only a verified actual cash-transfer event with the expected faction, player ID, exact amount, time after preparation and one unique active intent. Preserve source ID/raw text. Reject deposits, balance credits, duplicate evidence and ambiguous matches.

PAY-05. Deposits, personal faction balances, balance credits and actual cash transfers are distinct. A balance snapshot alone cannot establish that a war obligation was paid.

PAY-06. Reverse an incorrect ledger entry with a reason and pointer to the original. A local reversal never reverses a Torn transfer. Overpayments remain visible for leadership resolution.

EXP-01. CSV includes war/period, player ID/name, contribution, rule version, points, accrued/paid/due, currency, state and snapshot ID/time. Escape text fields against spreadsheet formula injection. Export values must equal the selected approved snapshot.

Pay in Torn: verified payment bridge

Verified route: Torn Wiki documents prefill of the recipient and amount in Give to User. This prepares a form only; it does not execute or verify a transfer. The payer needs an authenticated Torn session and Money Giving permission.

```
https://www.torn.com/factions.php?step=your
#/tab=controls&giveMoneyTo={player_id}&money={integer_amount}
```

BRIDGE-01. Offer Pay in Torn only for an approved positive remaining obligation and a current faction member. Show name, Torn ID, exact remaining amount, payment method and settlement revision. Build the link from validated integer ID/amount on an allowlisted torn.com route; never include a key.

BRIDGE-02. Give money reduces the member's recorded faction balance. Explain this before opening. It may create an unwanted balance effect for war pay. Pay Day does not change member balances; prefer it when that is the faction's payment policy.

BRIDGE-03. For Pay Day, open the known controls/Pay Day page and provide Copy player ID and Copy amount. Recipient/amount prefill for Pay Day is not verified, so do not promise it or invent URL parameters. Never silently replace cash payment with addMoneyTo balance credit.

BRIDGE-04. Create an expiring payment intent/claim before opening a form. Prevent another treasurer preparing the same obligation concurrently. Expiry permits a fresh check, not an assumption of failure. Show Prepared / Awaiting confirmation; opening a link never changes Paid.

BRIDGE-05. After opening/returning from Torn, poll every 30 seconds for up to 10 minutes, then retain Pending for scheduled reconciliation. A uniquely matched actual transfer atomically posts an idempotent payment and closes its intent. Timeout is not failure; a late payment can arrive. Recheck recipient/amount/revision before preparing a new link.

BRIDGE-07. Consume each source event once; check active intents across wars and previous allocations. Multiple matches or unclear text go to Needs review. Pending external transfers block replacement payments until checked. API downtime never implies success or failure. Stale external tabs cannot be invalidated by this app.

BRIDGE-06. For former members, retain the obligation but disable the faction-prefill route and offer manual settlement/evidence. Do not auto-send from a leader's wallet. Validate the prefill and Pay Day flow in an authorized session before production; this research did not submit transfers.

Source: https://wiki.torn.com/wiki/Faction#Give_To_User and #Pay_Day. Prototype must label external preparation separately from local demo recording.

Torn API mapping and access

Endpoint v2	Use
/faction/chains; /faction/{chainId}/chainreport	Chain history, aggregate contributions and milestones.
/faction/attacks	Detailed results, timestamps and ranked-war context.
/faction/members	Current roster; retain historical snapshots locally.
/faction/rankedwars; /faction/{rankedWarId}/rankedwarreport	War history, results, contributions and rewards.
/faction/chain	Current chain and timeout.
/faction/news?cat=giveFunds; /faction/balance	Reconciliation candidates and current balances.
/key/info	Available selections and faction permissions.

Detailed faction attacks and finances require Limited with faction API access or a suitable Custom key. These faction functions do not require keys from every member. attacksfull lacks some detailed classification fields and is not an equivalent replacement. Bonus rows have no timestamp; exact bonus time requires attack evidence.

Official API schema 6.15.0, reviewed 1 November 2026: https://www.torn.com/swagger/openapi.json

API-01. Before production, verify actual key selections, faction permissions, earliest recoverable detail, gaps and report reconciliation. The current server key was not accessed in this work.

API-02. Persist collected history. Pagination and from/to do not guarantee unlimited recoverable history. Aggregate reports and CSV may fill gaps only at their actual granularity.

Synchronization and source trust

DATA-01. Collect on the server with pagination and checkpoints. Upsert by external ID; overlap the latest range for late records and deduplicate. Never replace valid records with an empty failed response.

DATA-02. Store raw payload, endpoint, fetched_at, source time, parser version, batch and hash. Preserve UTC and expose source coverage. Never fabricate timestamps from aggregate totals.

DATA-03. Raw attacks are facts; chain reports are aggregates/fallback/reconciliation evidence. Do not add both for overlapping coverage. CSV without reliable IDs requires duplicate review before combining.

DATA-04. An entire-chain aggregate is not evidence for an arbitrary subperiod or hourly chart. Do not spread totals proportionally across missing timestamps.

DATA-05. States: complete, partial, aggregate-only, error and not synced. Show range, gaps, latest attempt and latest success. API configured does not establish data readiness.

DATA-06. Approval blockers include missing required classification, unknown recipient, unresolved duplicate, incomplete bonus coverage under an exclusion policy, ongoing war and unconfirmed budget. Aggregate mode permits only formulas fully supported by available fields.

DATA-07. Initial cadence target: 30-60 seconds during active chain/war; on-demand plus control sync for completed events. One server collection serves all viewers. Budget requests across the owner’s other applications.

Torn documents up to 100 requests/minute per user and request caching. Implement backoff/jitter, invalid/paused-key handling, API downtime and historic-read limits. Do not promise second-by-second freshness. Source: https://www.torn.com/api.html

Data model and nonfunctional requirements

Entities	Minimum invariants
Faction, User, Role	Server-enforced faction isolation and permission checks.
Member, MembershipSnapshot	Stable player ID, historical names and membership/exception coverage.
War, Chain, Attack, ImportBatch	External IDs, UTC intervals, source and unique attack identity.
PresetVersion, RuleSnapshot	Independent model/weights/scope/funding; immutable approved revision.
Income, FundMovement, Expense	Confirmed versus estimated money and reserve source.
Calculation, PayoutLine, Adjustment	Input hash, rule version, exact decimals, reason and actor.
PaymentIntent, Payment, Allocation, Reversal	Expiring preparation claim, immutable fact and explicit reversal.
AuditEvent, ReconciliationCandidate	Before/after, actor/time/request ID and unresolved evidence queue.
Quota, Shift, MilestonePlan, Alert	Independent participation goals and follow-on coordination.

TECH-01. Use whole-dollar money and decimal/rational intermediate arithmetic, not binary floating point for final payouts. Identical inputs and versions produce identical results.

TECH-02. Approve, revise, claim, allocate and reverse atomically with optimistic concurrency and idempotency. Sync never rewrites approved snapshots. Serialize competing preparations for one obligation.

TECH-03. Append-only audit trail; backup/restore procedure. Initial production targets: RPO 24 h and RTO 4 h. Back up V1 before migration. Redact credentials from logs.

TECH-04. Support 100 members/war; preserve calculation population across pagination. Keyboard focus, labelled inputs, readable contrast and 200% zoom; horizontal scrolling confined to tables on small screens.

Rounding and edge cases

ROUND-01. For a weighted pool, calculate exact shares, floor each and distribute remaining whole dollars by descending fractional remainder. Break ties by ascending player ID. Sorting/filtering must never change payouts.

ROUND-02. Bonuses are whole dollars. In fixed-rate mode, sum a player's exact components first, then round half up to whole dollars. Recheck the budget after rounding.

ROUND-03. Component displays explain the final amount. Include a rounding-adjustment line so displayed components sum to Final Payout; never round the total points before dividing.

Condition	Required behavior
Points = 0, variable fund > 0	Block division/approval; do not automatically divide equally.
Points = 0, variable fund = 0	Permit only fully funded bonuses or explicitly acknowledged zero settlement.
Negative income, fund, rate or weight	Reject. Adjustments use a separate permissioned record and reason.
Inside bonuses exceed fund	Block with exact deficit.
Missing respect/contribution	Unknown is not zero. Block if required by the rule.
Member left or changed name	Keep history/obligations keyed by player ID.
Duplicate import, payment or intent request	Return existing record/result; do not duplicate cash obligations.
Optional deductions	Disabled initially. If later introduced, apply after base, floor at zero and explicitly return withheld money; no hidden redistribution.

No automatic penalties, refunds or cross-war offsets. Base prototype implements positive bonuses, two calculation modes and deterministic rounding; it does not pretend a live transfer happened.

Acceptance tests

ID	Scenario and expected result
AT-01	Income 182,285,770 → reserves 18,228,577 each; player budget 145,828,616. Xanax expense 8,300,000 leaves player budget unchanged.
AT-02	W=160, R=1,019.52 and weights 1/0/0/0.1 → points 261.952, rate≈556,699.761788. Rounded 262 is never the divisor.
AT-03	Fund 100; A: W1/R10, B: W1/R0; weights W1/R0.1 → points 2/1, payouts 67/33.
AT-04	Fund 10; equal points for IDs10/20/30 → payouts4/3/3 regardless of row order.
AT-05	Fund100, equal points, inside bonus20 to A →60/40. Extra-funded bonus20 instead →70/50, total120.
AT-06	Fixed W3 × $10, fund20 → amount30, deficit10, approval blocked. No silent rate scaling.
AT-07	Changing a war's preset/weights leaves every other war and saved template unchanged. Discard restores the saved draft.
AT-08	Overlapping chain aggregates/raw attacks/reimported CSV never inflate payable totals. Unresolved duplicates block approval.
AT-09	Approved100, payment40 → due60. Retry is idempotent. Second payment60 → due0. Opening a Torn link leaves paid0.
AT-10	Approved snapshots survive sync. A lower revision than paid produces overpayment, not a negative transfer.
AT-11	Viewer mutations fail server-side. Concurrent stale approvals and competing payment claims are rejected.
AT-12	Incomplete required data blocks approval. Export equals the approved snapshot.
AT-13	Prefill uses validated ID and current remaining amount. Give money discloses member-balance effect. Pay Day never claims verified prefill.
AT-14	Former member/zero balance due/stale claim cannot prepare a faction-prefill payment. No action sends funds automatically.

Migration and implementation sequence

A. Establish trustworthy data

Inspect V1 server/schema/import behavior, back up, add stable IDs and coverage metadata. Reconcile one completed war against source attacks, ranked-war report and spreadsheet. Preserve aggregates at their real granularity and retain legacy unlinked events until mapped.

B. Add the leadership workspace

Build wars, players, chains, source readiness and quotas. Implement two calculation models, six presets, 10/10/80 reserves, reserve expenses and exact rounding. Run side-by-side with the existing spreadsheet before recording any production payment.

C. Add settlement controls

Individual leadership identities, approval snapshots, partial payments, intents/claims, audit and CSV. Test documented Give to User prefill and Pay Day manually in an authorized account. Add reconciliation assistance only after verified parsing/matching.

D. Add coordination

Watcher shifts, bonus assignment and alerts. Test delivery separately; Discord stays optional. Do not collect member credentials for private stats.

MIG-01. Preserve V1 functionality and required EN/ZH localization; new prototype and developer specification use English. Avoid mixed-language forms. Old data must remain recoverable after migration.

MIG-02. Source completeness and actual income are launch gates for production settlement, not for the local demo. The prototype is not a production backend, authorization implementation or Torn integration.

Functional design and prototype contract

Surface	Required working interactions
Overview	Select period/war; navigate to outstanding obligations and source issues.
Wars	Current-war entry; official-window preview; contributions, dollar-price explanation, calculation approval and sources only.
Calculator	Apply six presets; edit fund/income/weights/model; live preview; advanced scope; bonuses; save/discard; inspect breakdown.
Players	Search/filter/sort; player detail with contribution and payment history.
Chains	Full internal report: member contribution, bonus milestones, source coverage, search and chain-specific CSV.
Payments	Separate financial queue after calculation approval; individual form preparation, API-backed confirmation, partial payments, reversals and transfer history.
Sources / Settings	Visible Faction API connection panel, masked/disabled real-key input in demo, demo permission-check flow, source coverage and presets.

PROTO-01. Use clearly labelled demo data and local persistence with Reset demo. Include sample totals from the spreadsheet and a second independent war. Illustrative subtype/history detail is labelled demo. No private API keys, live money transfers or Discord messages.

PROTO-02. Use selected concept 1 and product name The Bonfire Faction Management. Match dark left navigation, white content, typographic budget summary, broad table and pale-teal rules panel. Correct visual-mock arithmetic in code.

PROTO-03. Payment dialog supports documented prefill and Pay Day fallback. External links only prepare a form. Simulate Torn transfer creates explicit demo API evidence; the matching algorithm then confirms it automatically. A timer alone must never fake payment. Explain balance effects and bonus funding.

DES-01. 14-16 px product text, tabular numerals, visible labels/focus, useful empty/error/unsaved/locked states. At 390 px, collapse navigation and rules layout; table scroll must not hide global actions.

Official war window and active war

TIME-01. Discover the faction's wars through rankedwars/current warfare selections, keyed by stable war ID and faction IDs. Read official start/end timestamps. Never infer a start or finish from the first/last attack, a date picked by an officer, target progress or elapsed local time.

TIME-02. Before official start, show Scheduled and the scheduled time. After start, without a valid final end, show In progress. A null/nonpositive end is not a real finish timestamp. Do not invent an end or allow final approval while the war is running.

TIME-03. When a positive end exists, fetch the completed rankedwarreport for that war ID and reconcile its interval/result. If sources disagree or the report is unavailable, show Finished - awaiting verification. Never overwrite conflicts silently.

TIME-04. After the report is verified, import the entire official window with pagination, deduplicate attack IDs and reconcile contribution totals. Show Finished - importing records until coverage is complete. Approval readiness is separate from war state and payment state.

TIME-05. Use attack ended timestamps within the agreed official window. Cross-boundary records or inconsistent war flags enter review. Changing a display time zone does not change timestamps, eligibility or existing approved snapshots.

Visible field	Behavior
Started / Scheduled start	Exact date and time from the war API, labelled TCT/UTC or selected zone.
Finished	Exact verified end, or Still in progress. Never show zero as a calendar date.
Duration	Actual end minus start; while running, explicitly show ongoing duration/unknown end.
Source & readiness	War ID, report verification, import coverage and latest successful sync.
Current war entry	Overview card and a Current war action in Wars. Detect automatically when connected; show no current war when none exists.

The prototype has an explicit current-war demonstration and an internal How are dates determined explanation. All demo intervals are labelled illustrative, including the August example: the screenshot provides a payout date, not the official war start/end.

Faction API connection and onboarding

KEY-01. Settings starts with a Faction API connection panel. A leadership-only Add faction key flow accepts an appropriate Limited/Custom key via HTTPS to the backend. No extra keys or setup from ordinary members. Do not collect a Torn account password.

KEY-02. Validate key identity, faction ID, current membership, allowed selections and faction permissions. Test required reads for wars/reports, detailed attacks, members and funds news. Show a precise missing-permission state and keep unsupported features unavailable.

KEY-03. Store the secret encrypted on the server and never in browser storage, client bundles, logs, analytics, CSV or generated links. Return only a masked identifier and connection status to the browser. Rotation/disconnection require authorized leadership and audit events.

KEY-04. Connection states: not connected, validating, connected, missing permissions, invalid/paused and temporary API failure. Show faction identity, last success, collection status and actionable recovery. A configured key alone never implies complete history.

KEY-05. After successful validation, discover the current war and recent completed wars, then enqueue bounded backfill and incremental collection. Display current war, official timestamps, source freshness and import progress. Preserve collected history if a key is revoked; stop new reads and identify the stale state.

KEY-06. The local prototype is a setup demonstration only. Its real-key input is disabled. Preview key setup runs explicitly labelled demo permission checks and enables a demo connection state; it never accepts or persists a real secret and never claims to have contacted Torn.

Additional acceptance checks

AT-15. Pool weight 1 is visibly labelled points and shows approximately $556,699.76 per war hit under the sample. Fixed mode uses $ inputs, starts from separate saved rates and never silently converts weight values into dollars.

AT-16. Approve calculation changes obligations/snapshot only; paid remains zero and no transfer intent is created. Payments does not duplicate the calculator. Each transfer is prepared individually.

AT-17. A chain opens internally with member totals summing to report hits/respect and bonus totals. Search does not change full-report totals. Unknown hit timestamps are labelled unavailable; no fabricated event times.

AT-18. An ongoing war has a visible entry, official-start field and no end; final approval is blocked. Switching TCT/UTC to Kyiv changes display only. Key setup is visible without accepting real credentials in the prototype.

Decisions and sources

Confirmed by the product owner

Existing tool is V1; leadership audience; flexible per-war rules and presets; 10/10/80 allocation; free Xanax charged to faction reserves; no extra ordinary-player keys/actions; concept 1; exact name The Bonfire Faction Management; English requirements; payment-form preparation and automatic API-backed transfer confirmation.

Defaults proposed for implementation

Weights 1/0/0/0.1; all-hit respect with bonuses included; successful-result classification; time-window boundaries; largest-remainder allocation; no automatic penalties; individual leadership identities for mutations; automatic API confirmation with manual review for unresolved evidence. Validate these policy details before production.

Production checks still required

Full spreadsheet formulas, actual respect scope, unusual attack classification, reward-sale income, authorized staff, real key permissions/history coverage, Give to User prefill in the actual faction, Pay Day behavior and transfer-news reconciliation. None of these was tested through a live transfer.

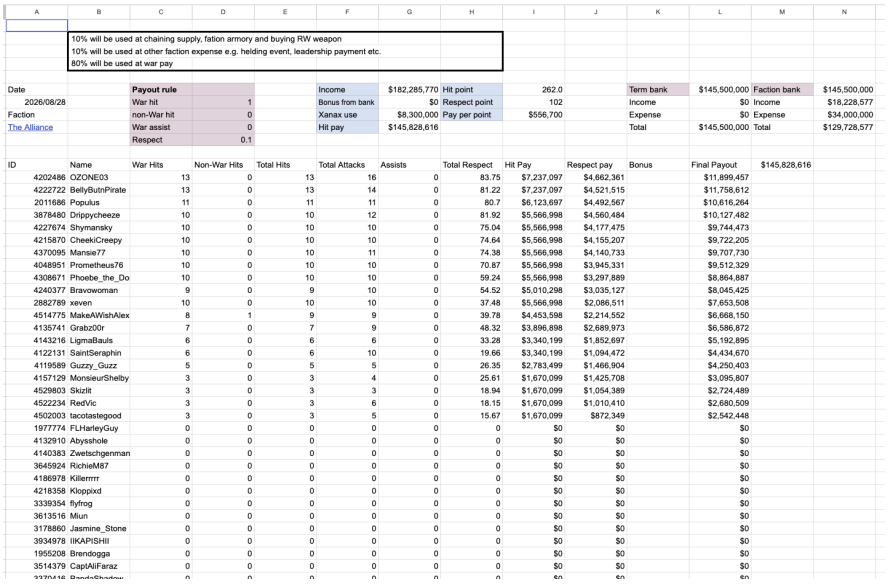
S1. User-provided payout-sheet image for 28 August 2026; original workbook unavailable. Sample arithmetic was reconstructed from visible rows.

S2. Existing read-only interface: torn-chain-tracker.onrender.com. No production changes; repository/backend not supplied.

S3. Official API schema 6.15.0: www.torn.com/swagger/openapi.json.

S4. Official API rules, caching/access/limits: www.torn.com/api.html.

S5. Official payment form, prefill parameters and balance distinction: Torn Wiki / Give To User and Pay Day. Reviewed 1 November 2026.



10% will be used at chaining supply, fation armory and buying RW weapon
10% will be used at other faction expense e.g. helding event, leadership payment etc.
80% will be used at war pay

Date		Payout rule			Income	$182,285,770	Hit point	262.0		Term bank	$145,500,000	Faction bank	$145,500,000
2026/08/28		War hit	1		Bonus from bank	$0	Respect point	102		Income	$0	Income	$18,228,577
Faction		non-War hit	0		Xanax use	$8,300,000	Pay per point	$556,700		Expense	$0	Expense	$34,000,000
The Alliance		War assist	0		Hit pay	$145,828,616				Total	$145,500,000	Total	$129,728,577
		Respect	0.1										

ID	Name	War Hits	Non-War Hits	Total Hits	Total Attacks	Assists	Total Respect	Hit Pay	Respect pay	Bonus	Final Payout	$145,828,616
4202486	OZONE03	13	0	13	16	0	83.75	$7,237,097	$4,662,361		$11,899,457	
4222722	BellyButnPirate	13	0	13	14	0	81.22	$7,237,097	$4,521,515		$11,758,612	
2011686	Populus	11	0	11	11	0	80.7	$6,123,697	$4,492,567		$10,616,264	
3878480	Drippycheeze	10	0	10	12	0	81.92	$5,566,998	$4,560,484		$10,127,482	
4227674	Shymansky	10	0	10	10	0	75.04	$5,566,998	$4,177,475		$9,744,473	
4215870	CheekiCreepy	10	0	10	10	0	74.64	$5,566,998	$4,155,207		$9,722,205	
4370095	Mansie77	10	0	10	11	0	74.38	$5,566,998	$4,140,733		$9,707,730	
4048951	Prometheus76	10	0	10	10	0	70.87	$5,566,998	$3,945,331		$9,512,329	
4308671	Phoebe_the_Do	10	0	10	10	0	59.24	$5,566,998	$3,297,889		$8,864,887	
4240377	Bravowoman	9	0	9	10	0	54.52	$5,010,298	$3,035,127		$8,045,425	
2882789	xeven	10	0	10	10	0	37.48	$5,566,998	$2,086,511		$7,653,508	
4514775	MakeAWishAlex	8	1	9	9	0	39.78	$4,453,598	$2,214,552		$6,668,150	
4135741	Grabz00r	7	0	7	9	0	48.32	$3,896,898	$2,689,973		$6,586,872	
4143216	LigmaBauls	6	0	6	6	0	33.28	$3,340,199	$1,852,697		$5,192,895	
4122131	SaintSeraphin	6	0	6	10	0	19.66	$3,340,199	$1,094,472		$4,434,670	
4119589	Guzzy_Guzz	5	0	5	5	0	26.35	$2,783,499	$1,466,904		$4,250,403	
4157129	MonsieurShelby	3	0	3	4	0	25.61	$1,670,099	$1,425,708		$3,095,807	
4529803	Skizlit	3	0	3	3	0	18.94	$1,670,099	$1,054,389		$2,724,489	
4522234	RedVic	3	0	3	6	0	18.15	$1,670,099	$1,010,410		$2,680,509	
4502003	tacotastegood	3	0	3	5	0	15.67	$1,670,099	$872,349		$2,542,448	
1977774	FLHarleyGuy	0	0	0	0	0	0	$0	$0		$0	
4132910	Abysshole	0	0	0	0	0	0	$0	$0		$0	
4140383	Zwetschgenman	0	0	0	0	0	0	$0	$0		$0	
3645924	RichieM87	0	0	0	0	0	0	$0	$0		$0	
4186978	Killerrrrr	0	0	0	0	0	0	$0	$0		$0	
4218358	Kloppixd	0	0	0	0	0	0	$0	$0		$0	
3339354	flyfrog	0	0	0	0	0	0	$0	$0		$0	
3613516	Miun	0	0	0	0	0	0	$0	$0		$0	
3178860	Jasmine_Stone	0	0	0	0	0	0	$0	$0		$0	
3934978	IIKAPISHII	0	0	0	0	0	0	$0	$0		$0	
1955208	Brendogga	0	0	0	0	0	0	$0	$0		$0	
3514379	CaptAlFaraz	0	0	0	0	0	0	$0	$0		$0	

Source thumbnail only. Use the exact values on page 5, not rounded labels in the image, when implementing calculations.